

Capstone Project

Thick-Client, Application-Class, Local-First SPA (REST Client)

Overview

Deliver a **thick-client Single-Page Application (SPA)** in which the primary **computation, rendering, and state management** execute in the browser. Your application must **integrate a rendering stack**, act as a **real REST client** using `fetch` with `async/await` for **public reads** and **cloud writes**, apply an **object-oriented, modular design** appropriate to an application-class codebase, and **persist state locally** while supporting **cloud synchronization**. The finished product should feel like real software—**responsive, supporting multiple views or complex viewport rendering (Canvas/WebGL)**, and **easy to understand** from its documentation.

Tracks (Types of Apps by Usage)

Pick **one** track for your project. The architectural requirements (rendering stack, REST client, OO + modules, local-first + cloud sync) apply to **all** tracks.

Productivity

Apps that **organize, create, configure, or practice**. Formats may be **editors** (text/visual/canvas), **planners/boards**, **configurators/builders**, or **trainers/practice tools**. Examples include **documents**, **kanban boards**, **presets/projects**, and **exercise sets**. They must be **stateful, interactively responsive**, accept **user input** with **system updates** (e.g., live validation, previews, recalculation), and support **persistent state management**—the app remembers **drafts, versions, presets, or progress** so work can be resumed or shared.

Analytics

Apps that **explore, monitor, or explain** data. Formats may be **dashboards/inspectors**, **maps**, **explorable explainers**, or **notebook-style stories**. They can present data **tabularly or visually**, support **filter/sort/search**, and perform **spatial or time-series** analysis—including **plots, animations, trend lines, clustering**, and other visual encodings. Domains are open—**fitness, health, finance, movies, social networks, current events**, and more. They must be **stateful, interactively responsive**, accept **user input** with **system updates** (e.g., recompute, re-render, cross-highlight), and support **persistent state management**—the app remembers **saved views, filters, presets, or annotations** so a viewpoint can be reopened or shared.

Games / Entertainment

Apps that **simulate, play, challenge, or narrate**. Formats may be **turn-based or real-time, graphical (Canvas/WebGL) or textual** (e.g., interactive/branching fiction, chat-fiction). Examples include **strategy/simulation**, **puzzles**, **arcade loops**, and **branching stories**. They must be **stateful, visually or interactively responsive**, accept **user input** with **system updates**, and support **persistent state management**—the game remembers something from prior playthroughs (at minimum a **best/high score**; ideally **saved sessions** that can be resumed).

Choose **one** and state it in both `README` and `/docs/pitch.md`

Architecture

This project is a **thick-client SPA**. It runs in a single HTML shell with **client-side routing or a view manager** and is organized into **ES modules**. The design must include **at least three meaningful classes** with clear responsibilities (e.g., **models**, **controllers/systems**, **renderers/views**) and **at least six modules** at the top level of the codebase. The **browser is the primary runtime**: validation, state transitions, and error handling execute client-side rather than being offloaded to a backend.

Minimum architectural expectations

- **Routing/View composition**: client-side router or explicit view manager.
- **Modules**: ES modules (`type="module"`) with ≥ 6 top-level modules (e.g., `state/`, `services/`, `ui/`, `engine|viz/`, `routes/`, `utils/`).
- **OO structure**: ≥ 3 domain classes with clear roles (model, system/controller, renderer/view).
- **Rendering stack integration**: DOM/Components (**Vanilla**, **Web Components**, **Lit**, **React**, **Vue**) or Canvas/WebGL (**Phaser**, **Pixi**, **p5**, **regl**) or Viz/Maps (**D3**, **Plotly**, **Chart.js**, **ECharts**, **uPlot**, **Leaflet**, **MapLibre**). Your code **owns orchestration** (composition/inheritance, state/data flow, effect lifecycles)—not just one-off helper calls.

Rendering Integration

Select **at least one of these rendering substrates** for your app and integrate it deeply. Your code must **own orchestration** (composition/inheritance, state/data flow, effect lifecycles)—not just wire a few helpers.

A) DOM / Component Frameworks

- **Eligible**: React, Vue, Web Components, Lit, Vanilla DOM, Alpine, Petite-Vue
- **Use when**: the UI is **component-heavy** (forms, panels, multi-view SPA) and you need **structured state** → **view** flows.
- **Expectations**: reusable components; clear **props** → **render**; controlled inputs; local state/effects with documented data flow (and a central store if used).

B) Canvas / WebGL

- **Eligible**: Phaser, HTML Canvas, PixiJS, p5, regl, ThreeJS
- **Use when**: you need **real-time rendering**, **pixel/scene control**, or **game/simulation loops** (physics, hit-testing, sprites, particles).
- **Emphasis**: render loop; scenes/layers; physics/hit-testing; HUD composition.

C) Visualization / Mapping

- **Eligible**: D3 (with Canvas as needed), Observable Plot, Chart.js, ECharts, Plotly, uPlot, Leaflet, MapLibre
- **Use when**: the product is **charts/maps first**—**filters**, **brush/zoom**, **linked views**, and clear encodings of **time/space**.
- **Emphasis**: scales; interactions (brush/zoom); synchronized views/layers; cross-highlighting (prefer Canvas/WebGL for large datasets)

Networking & REST Responsibilities

Your app must integrate with **external web services** using `fetch` with `async/await` to demonstrate real HTTP reads and writes.

- **Public read (GET):** at least **one** GET to an **open API**, a **published Google Sheet** (CSV/GViZ), or **static JSON** you host.

Design caveat: only fetch hosted JSON for **non-trivial reasons** -- e.g., data is **updated routinely**, is **too large** to bundle, or must be **shared/controlled** outside the app. If the data is **stable** and small, **package it with the app** instead of fetching it.

- **Cloud write (PUT/POST):** at least **one** write of **non-sensitive public artifacts** (e.g., views, presets, results, projects) to JSONBin.

Design caveat: If user-facing cloud saves aren't appropriate: use the cloud write for **lightweight usage analytics** (e.g., increment a counter for app opens or completed sessions). Keep this **aggregate, anonymous**, and **documented** in your README (what you track and why).

What belongs on the client vs. the cloud

- **Client (local-first, personal):** drafts, user settings, recent items, runs/progress/high scores, saved views/filters that are **personal**. Persist in `localStorage` (or IndexedDB). The app should **relaunch and restore** without the network.
- **Cloud (shared, public):** **leaderboards, public presets, daily challenges/configs**, or **aggregate usage counts** intended for **all users**. Use JSONBin for **non-sensitive** data only.

Local-First Behavior

Treat the app like a **desktop application that runs in the browser**: it should remember user state **between sessions** using only **client-side storage**. All meaningful computations occur within the client.

- **Persist locally:** save session/state to `localStorage` (or IndexedDB if larger or structured). On relaunch, **restore** and render immediately.
 - **What to remember (examples):** drafts/documents, views/filters, presets, runs/progress/high scores, last opened item, user settings (theme, accessibility), recent files.
 - **Keying & schema:** use clear keys and a small JSON schema; include a **version** field so you can migrate later.
 - **Autosave & timing:** autosave on meaningful changes.
 - **Export/Import (recommended):** provide a simple **Export JSON / Import JSON** flow so users can back up or move their data without any server.
-

Interactivity and Responsiveness

Design for **meaningful interaction**, not static screens.

- **Interactivity:** implement **multiple input paths**, wired into your model/state. Examples include **button bindings**, **mouse/touch clicks or gestures**, **keyboard controls/shortcuts**, **form inputs**, **timers/real-time loops** (for games/sims), or **controller-style** inputs. User actions must trigger **visible system updates** (state change → re-render).
 - **Responsiveness:** primary views should remain usable on **laptop** ($\geq 1280 \times 720$) and **tablet** ($\geq 768\text{px}$) widths. Complex viewports (Canvas/WebGL/charts) should **resize gracefully**.
 - If your app targets a **specific input/device** (e.g., keyboard + landscape) and is not designed for others, **detect unsupported contexts** and show a clear **“Not supported”** or **“Requirements”** screen (e.g., “Requires keyboard in landscape,” “Touchscreen required”).
 - **Feedback & states:** for long or async actions, show **loading / empty / success / error** states; disable or guard actions while busy; keep messages concise.
-

Project Timeline & Roadmap

1-week (21-day) Sprint-Cycle:

- Days 1–2: Sprint 1 deliverables + board
 - Days 3–4: Sprint 2 (MVP vertical slice).
 - Days 5–7: Sprint 3 (Full + polish + demo).
-

Sprint 1 — Pitch & Roadmap Proposal (2 days)

Purpose

Lock scope, prove feasibility, and stand up the plan so Sprint-2 coding can start immediately.

Deliverables

- **One-pager pitch** (</docs/pitch.md>): track, product sentence, problem/user, core loop (3–5 sentences).
- **Demo inspiration audit**: 3–5 sentences on what runs **in the client** (rendering, state, interaction) and how you'll add **REST (GET) + local-first + JSONBin** to reach application-class.
- **Public data source worksheet**: named **public GET** (or published CSV/JSON), tiny sample request/response, **HTTPS + CORS** confirmation, **cache TTL**.
- **JSONBin plan**: schema of what you will **PUT/POST**, example payload, **merge policy** (e.g., [updated_ts](#) last-write-wins).
- **Architecture sketch** (/docs/architecture_sketch.md): SPA router/view manager, state/store, services (publicApi/cloud/local), planned **OO classes** (≥ 3), and the **rendering stack**.
- **Roadmap** (</docs/roadmap.md>): **MVP (Sprint-2)** vs **Full (Sprint-3)**; top 3 risks + mitigations.

Definition of Done (</docs/dod-sprint1.md>)

- Pitch, worksheet, sketch committed under </docs/> (board link in README).
 - Architecture sketch shows ≥ 6 **modules** you intend to implement.
 - JSONBin bin (or placeholder) created; **schema** documented.
-

Sprint 2 — MVP Vertical Slice

Purpose

Ship the smallest slice that exercises the full system end-to-end.

Deliverables

- **Running SPA skeleton:** single HTML shell, **router/view manager**, **central store**, basic layout.
- **Vertical slice flow** (one path):
 1. User input → **state update**
 2. **Render** via the chosen rendering stack
 3. **Public GET** with `async/await` + `loading/empty/error` + **TTL cache**
 4. **Cloud write** to JSONBin (**PUT/POST**) for a small artifact (e.g., `view/preset/run`)
 5. **Local-first boot:** read from `localStorage`, then merge network
- **OO realization:** implement ≥ 3 **classes** with clear responsibilities (e.g., `Dataset/FilterModel/ChartView` or `Scene/Player/CollisionSystem`).
- **Resilience basics:** retry with **backoff** for transient errors; **AbortController** on route/view changes.
- **MVP README section:** run steps; endpoints with tiny request/response samples; JSONBin schema; **merge policy**; what the slice demonstrates.

Definition of Done (`/docs/dod-sprint2.md`)

- From a fresh clone, dev server runs the app (`npm run dev` or `http-server`).
 - **30–60s GIF** shows local boot → GET → render → PUT/POST.
 - Console free of uncaught errors; visible error states.
 - `/src/` contains ≥ 6 **ES modules** and the implemented classes.
-

Sprint 3 — Feature-Complete & Polish

Purpose

Complete planned features, deepen interaction, and finalize docs/release.

Deliverables

- **Full feature set** from Sprint-1 roadmap (non-MVP items now present). In your **DoD**, list each enhancement with acceptance checks.
- **Interactivity depth**: implement **multiple input paths** (e.g., buttons, keyboard controls/shortcuts, mouse/touch gestures, forms, timers/loops) wired into model/state (not cosmetic).
- **Responsiveness**: primary views usable at **laptop ($\geq 1280 \times 720$)** and **tablet ($\geq 768\text{px}$)**. Complex viewports (Canvas/WebGL/charts) **resize gracefully**. If your app targets a specific input/device (e.g., keyboard + landscape), **detect unsupported contexts** and show a clear “**Not supported / requirements**” screen.
- **Final README**: architecture overview, class roles, module map, routing/state flow, endpoint/spec snippets, JSONBin schema, TTLs, merge policy, input patterns, run/deploy instructions.
- **Deployment**: GitHub Pages (or bullet-proof local run steps).

Definition of Done (/docs/dod-sprint3.md)

- Sprint-1 feature list marked complete; deviations noted.
-

Documentation

Treat docs like part of the product. Your repo should have a **public-facing README.md** that reads like an app landing page (what it is, how to use it, how it works at a high level, demo links) and a **developer docs area in docs/** that contains all planning and course artifacts (pitch, roadmap, architecture sketch, endpoints/schema details, and per-sprint DoDs). Keep JSON examples in the README tiny and link to full details in `docs/`. Store screenshots/GIFs under `docs/media/`. Never commit secrets; any endpoints shown should be HTTPS/CORS-verified and non-sensitive. This split keeps the repository professional for the public while making grading straightforward.

Public README (README.md)

Use this as a landing page for users and reviewers (no “homework” wording).

Recommended order

- 1. Product Name & One-Line Summary**
A short value statement: what it is and why it exists.
- 2. Features**
4–6 bullets that describe capabilities and key interactions (keep it user-oriented).
- 3. Screenshots / Demo**
 - 1–2 screenshots or a short GIF from `docs/media/`
 - Demo video (60–120s)** link
- 4. Live Demo / Install & Run**
 - Live demo (GitHub Pages):** `https://<your-org>.github.io/<repo>/`
(For SPAs, use hash routing or a `404.html` redirect so deep links work.)
 - Local (fallback):** `npm i && npm run dev` or `npm http-server .`
 - Requirements:** e.g., “Requires keyboard in landscape,” “Designed for 1280×720+.”
- 5. How It Works (High-Level)**
 - Rendering stack** (DOM/Components, Canvas/WebGL, or Viz/Map) + one-line rationale
 - Architecture in brief:** key classes/modules and how views update from state
 - Local-first behavior:** what persists between sessions and what restores on launch
- 6. Data & Networking (High-Level)**
 - Public GET:** endpoint name + 1 tiny response snippet
 - Cloud write (JSONBin):** what’s written + 1 tiny payload snippet
 - Note: non-sensitive data only; reads are cached with a short TTL
- 7. Configuration (Optional)**
Any app settings or flags users can toggle.
- 8. License / Credits**
Asset licenses, third-party libraries, and acknowledgments.
- 9. Developer Docs (Links)**
Short list linking to detailed docs (below), e.g.:
`docs/architecture_sketch.md • docs/endpoints.md • docs/jsonbin_schema.md • docs/roadmap.md • docs/pitch.md • docs/dod-sprint1.md • docs/dod-sprint2.md • docs/dod-sprint3.md`

Development Docs (/docs/)

All planning artifacts and course deliverables live here.

Suggested layout

```
docs/  
  pitch.md  
  roadmap.md  
  architecture_sketch.md  
  jsonbin_schema.md  
  dod-sprint1.md  
  dod-sprint2.md  
  dod-sprint3.md  
  media/  
    screenshot-1.png  
    screenshot-2.png  
    demo.mp4
```

File templates (copy/paste)

docs/pitch.md

```
# Pitch  
**Track:** Productivity / Analytics / Games  
**Product (one line):** What it is and why it exists.  
**Problem & User:** Who it helps and the need.  
**Core Loop (3-5 sentences):** User action → state change → render → outcome.
```

docs/roadmap.md

```
# Roadmap  
## MVP (Sprint 2)  
- Feature A ...  
- Feature B ...  
- Vertical slice path: input → state → render → GET → PUT  
  
## Full (Sprint 3)  
- Enhancement 1 ...  
- Enhancement 2 ...  
  
## Risks & Mitigations (Top 3)  
1) Risk ... → Mitigation ...  
2) ...  
3) ...
```

docs/architecture_sketch.md

```
# Architecture Sketch  
  
Add an ASCII diagram or image plus brief legend. Include:  
• View composition / routing (router or view manager)  
• Top-level module map (≥6 modules): state/, services/, ui/, engine|viz/, routes/, utils/  
• Core classes (≥3) and responsibilities (model, system/controller, renderer/view)
```

docs/jsonbin_schema.md

```
# JSONBin Schema & Merge Policy
## Schema
- Document shape ...
- Key fields ...

## Examples
PUT payload (truncated): ...
Result (truncated): ...

## Merge Policy
- Last-write-wins (timestamp) per item, de-dupe by id
- Or per-record versioning when domain warrants it
```

docs/dod-sprint1.md

```
# Sprint 1 – Definition of Done (Planning)
- [ ] Roadmap set (MVP vs Full) in `docs/roadmap.md`
- [ ] Architecture sketch matches stack & module plan (≥6 modules, ≥3 classes)
- [ ] Public GET verified (paste a tiny JSON snippet or screenshot)
- [ ] JSONBin bin (or placeholder) + schema documented
- [ ] Project board created with 8–12 Sprint-2 issues (title, AC, labels, size, owner)
- [ ] `docs/` committed; board link added to README
```

docs/dod-sprint2.md

```
# Sprint 2 – Definition of Done (MVP Vertical Slice)
- [ ] App boots locally; vertical slice: input → state → render → GET → PUT
- [ ] UI states for network: loading / empty / error
- [ ] Local-first boot: reads localStorage; background refresh + merge
- [ ] ≥6 ES modules; ≥3 classes implemented
- [ ] Evidence: GIF (docs/media), board filter link, tag/commit, run instructions updated
```

docs/dod-sprint3.md

```
# Sprint 3 – Definition of Done (Full + Polish)
- [ ] Full feature set delivered (each enhancement listed here with AC)
- [ ] Interaction depth (two+ input modes) and resized viewports behave well
- [ ] Final README: architecture, data flow, endpoints, TTLs, merge policy, run/deploy
- [ ] Demo video (60–120s) linked; deploy URL or verified local run
- [ ] Evidence: board filter link, release tag, media assets
```

Tone & review checklist

- **README** reads like a product page (features, demo, how it works, how to run).
- **docs/** holds pitch, roadmap, architecture, endpoints, schemas, DoDs, board links.
- No secrets in the repo; endpoints shown with small examples.
- Screenshots/GIFs stored under **docs/media/** and linked where needed.

Grading Rubric (Total 100 pts + up to 5 bonus)

Sprint 1 — Pitch & Roadmap Proposal (20 pts)

- ☐ **Track, Product, Core Loop (4 pts)**
Clear one-pager: chosen track, product sentence, problem/user, 3–5 sentence loop.
- ☐ **Public Read Plan (GET) (4 pts)**
Named endpoint (or published CSV/JSON), HTTPS+CORS verified, tiny sample resp, TTL stated.
- ☐ **Cloud Write Plan (JSONBin) (4 pts)**
Schema + example payload; merge policy (e.g., timestamp LWW or per-record version) written.
- ☐ **Architecture Sketch (4 pts)**
SPA router/view manager, state/store, services (publicApi/cloud/local), planned **≥3 classes, ≥6 modules**.
- ☐ **Roadmap & Board (4 pts)**
MVP vs Full features, top 3 risks; project board with **8–12 Sprint-2 issues** (title, AC, labels, size, owner).

Sprint 2 — MVP Vertical Slice (40 pts)

- ☐ **Thick-Client Execution (8 pts)**
App boots; one vertical slice: **input** → **state** → **render** in the client (no server UI).
- ☐ **Rendering Integration (8 pts)**
Deep use of chosen stack (DOM/Components, Canvas/WebGL, or Viz/Map); code **owns orchestration**.
- ☐ **Networking & REST (8 pts)**
`fetch` + `async/await`; **public GET** with **loading/empty/error** + **TTL**; **JSONBin PUT/POST** for a small artifact.
- ☐ **Local-First (8 pts)**
Restore from localStorage; background refresh; deterministic merge behavior visible (policy referenced).
- ☐ **OO & Modularity (8 pts)**
≥3 meaningful classes, ≥6 ES modules; responsibilities coherent.

Sprint 3 — Feature-Complete & Polish (40 pts)

- ☐ **Feature Completion (18 pts)**
Non-MVP features from roadmap delivered; each enhancement appears in DoD with AC.
- ☐ **Interactivity & Responsiveness (12 pts)**
Multiple input paths (buttons, keyboard, mouse/touch, forms, timers/loops) wired to state; complex viewports resize; unsupported contexts show a clear “requirements” screen when applicable.
- ☐ **Runability & Public README (10 pts)**
Product-facing README (features, demo, how-it-works, run steps, tiny endpoint/snippet); docs/ linked; Pages deploy or bullet-proof local run; demo video (60–120s).

Bonus — Showcase Bonus (+10 pts)

Projects that are deemed outstanding may be eligible for a showcase bonus and shared as the idealized example for this project in future courses.

Appendix: Example Submission (based on Lab 5)

/docs/pitch.md

```
# Title

**Dodger – Phaser (JS) Registry Edition** *(Games Track)*

## Product

A **2D browser-based dodger game**. Instant play in any modern browser—just open a URL and go. Keyboard controls only; short, escalating runs; fast restarts.

## Problem & User

**Casual gamers** who want a no-install arcade experience that's **fast, fun, and addictive**, playable on any laptop with a browser. They prefer tight control, quick play sessions, and immediate replays without accounts or setup. Something that can be enjoyable in **3-minute increments** but highly replayable.

## Core Loop (3-5 sentences)

Use the arrow keys to dodge enemies that spawn from the right and move left. Enemy spawn rate and speed increase over time; collision triggers a brief screen flash and restarts the scene. The game keeps **session-level state** (score, top score, winner) in the **Phaser Registry** so scenes and the HUD can share values without prop-drilling. Views: **Title/Play** only for MVP; **Title → Play → (Game Over)** for the full version with scene management.

## Why Phaser & Registry (grounded in the lab)

* **Phaser scene lifecycle** (`preload` → `create` → `update`) drives all rendering/physics locally in the browser.
* **OOP classes** extend `Phaser.Scene` and `Phaser.Physics.Arcade.Sprite` for Player/Enemy/Projectile/PowerUp.
* **Registry** is the shared data spine for scene-to-scene state (e.g., `top_score`, `winner`).
```

/docs/roadmap.md

```
# Sprint 1 – Setup & Planning (Roadmap Only)

**Goal:** define the work for Sprints 2-3 and get the repo runnable.

## Deliverables (concise)
- **Repo boots** (`index.html`, `assets/`, `src/`; run via `npx http-server` or `npm run dev`).
- **MVP chosen** (the vertical slice for Sprint 2) and **Full list** (Sprint 3 features).
- **Roadmap** (`/docs/roadmap.md`): MVP list • Full list • Top 3 risks.
- **Project board** created with **8-12 Sprint-2 issues** (titles + brief AC + S/M + assignee) and columns: *Backlog • Sprint 2 • Sprint 3 • Done*. Link the board in README.

# MVP (Sprint 2) – Lab Goals 1-8

* **Goal 1-2:** project set-up; stub `PlayScene`; add to `game.js` scene array.
* **Goal 3:** background.
* **Goal 4-5:** `Player` sprite + input + physics, `move()` called from `update()`.
* **Goal 6-7:** `Enemy` class; timed spawns; leftward motion with varied speeds.
* **Goal 8:** `this.physics.add.overlap(player, enemies, game_over)`; camera flash; `scene.restart()`.

---

# Full Version (Sprint 3) – Enhancements 1-8

- Hitbox tuning
- Scrolling background (`TileSprite`)
- Simple animations
- **Top Score challenge** (HUD shows `score`, `top_score`, `winner`)
- **Projectiles** – player
- **Projectiles** – enemy
- **Power-ups** – *Slay*
- **Power-ups** – *Projectile*
- **Title Screen & Scene Management** (Registry persists `top_score`/`winner` across restarts)

---

# Risks & Mitigations

* **HUD desync:** use **Registry** + `changedata` handlers (not polling).
* **Spawn spikes:** cap active enemies; pace via timers.
* **Performance:** pool enemies/projectiles; avoid per-frame allocations.
```

/docs/architecture_sketch.md

```

project/
├── index.html           # loads phaser.js → classes → scenes → game.js (in order)
├── assets/              # background.png, player.png, enemy.png, etc.
└── src/
    ├── phaser.js        # provided library (loaded first)
    ├── game.js          # creates Phaser.Game with config + scene list
    ├── PlayScene.js     # MVP: preload/create/update; helpers: create_* / update_*
    ├── TitleScene.js    # Enhancement 8: title, instructions, start key
    ├── Player.js        # extends Arcade.Sprite; input, move(), attack()
    ├── Enemy.js         # extends Arcade.Sprite; leftward velocity; attack() (enh.)
    ├── Projectile.js    # extends Arcade.Sprite; velocity set on create
    └── PowerUp.js       # base + SlayPowerUp/ProjectilePowerUp subclasses

```

/docs/dod-sprint1.md — Planning & Proposal (example)

```

# Sprint 1 – Definition of Done (Planning & Proposal)

- ✓ Roadmap set: MVP (Lab 05 Goals 1-8) + Enhancements listed in /docs/roadmap.md.
- ✓ Architecture (lab-aligned): plain JS + script order + Phaser Registry noted; files listed.
- ✓ Project board: Backlog / Sprint-2 / Sprint-3 / Done; 8-12 Sprint-2 issues with title, AC, labels, size, owner.
- ✓ Docs wired: /docs/* committed; board link in README.

```

/docs/dod-sprint2.md — MVP Vertical Slice (example)

```

# Sprint 2 – Definition of Done (MVP Vertical Slice)

## A. Acceptance Checklist
- ✓ Boots from `index.html` in plain JS (script order correct); game starts.
- ✓ Core loop: Player moves; enemies spawn; collision triggers restart (Lab 05 Goals 1-8).
- ✓ HUD shows live score/time; HUD reacts to **Phaser Registry** updates.
- ✓ Performance guardrails: pooled/capped enemies; no per-frame allocations in hot loops.
- ✓ README updated with MVP section + 20-30s GIF (controls, scene flow, registry keys, run steps).

## B. Evidence
- Project board (filtered to **Sprint-2**): <link to board with filter or view>
- Closed issues (label: `sprint-2`): <link to GitHub issues query> (e.g., **10 closed**, 2 added during sprint)
- Demo GIF: `/docs/media/mvp.gif`
- Release tag/commit: `v0.1-mvp` → <link to tag/commit>
- Deploy/Run: GH Pages <url> (or `npx http-server` steps in README confirmed)

## C. Variance & Notes
- Added: `MVP: cap active enemies` after we saw frame dips.
- Removed: none.
- Risk: spawn spikes on low-end laptops → mitigated by pooling + cap at 30 enemies.
- Next sprint: integrate TitleScene + power-ups; begin polish pass.

```

/docs/dod-sprint3.md — Feature-Complete & Polish

```

# Sprint 3 – Definition of Done (Full + Polish)

## A. Acceptance Checklist
- ✓ Enhancements implemented (commit to 2-4): animations, enemy variety, power-ups, Title/Registry top score.
- ✓ Interaction depth: at least two of {pause, keyboard-only menu, power-ups, tuned ramp, particles}.
- ✓ Accessibility: keyboard-reachable menus; readable HUD (contrast/size); works on laptop/tablet widths.
- ✓ Performance pass: pooled sprites; no obvious jank; console clean.
- ✓ Final README: architecture/file map (plain JS), scene responsibilities, registry keys, polished run steps.
- ✓ Demo video (60-120s) recorded and linked.

## B. Evidence
- Project board (filtered to **Sprint-3**): <link>
- Closed issues (label: `sprint-3`): <link> (e.g., **12 closed**, 3 added during sprint)
- Release tag/commit: `v1.0-full` → <link>
- Deploy/Run: GH Pages <url> (or verified local steps)

## C. Variance & Notes
- Added: `Polish: camera shake on collision`; `A11y: high-contrast HUD`.
- Deferred: `Enemy projectiles` (descope due to time).
- Lesson: moving pooling to a factory cut GC spikes and stabilized 60fps.

```

Instructor notes (what's missing to meet the capstone bar)

This Sprint-1 sample mirrors the **lab** (no persistence across sessions; no web services). To satisfy the **Thick-Client, Application-Class, Local-First SPA (REST Client)** requirements later, students must add:

1. **Between-session persistence:** a small `localStorage` adapter (save `top_score`, `winner`, recent runs).
2. **REST client:** a JSONBin client to **PUT/POST runs** and **GET** a shared leaderboard (with loading/empty/error, TTL cache, retry/backoff, `AbortController`).
3. **(Optional) Daily config:** public `daily.json` seed/params (keyless GET), cached with TTL; merge with registry at boot.

Those are **not** included here by design -- this Sprint-1 sample is a faithful reflection of the **JS Phaser lab** so students can recognize it and submit their own planning docs accordingly.

Appendix: Demo Thick Web Client Applications

Productivity (editors / whiteboards / diagramming)

- **Excalidraw** – collaborative drawing/diagramming in the browser. Great example of a canvas-driven editor. ([Excalidraw](#))
- **tlldraw** – modern infinite-canvas whiteboard; the **examples** page has many mini-demos. ([tlldraw](#))
- **diagrams.net (draw.io)** – full diagram editor running entirely in the browser. ([diagrams.net](#))
- **Photopea** – Photoshop-style raster/vector editor, fully browser-resident. A great “application-class” example. ([Photopea](#))

Analytics (charts / data stories / maps)

- **Observable Plot Gallery** – dozens of data-vis examples students can click and explore. ([Observable](#))
- **D3 Gallery (Observable)** – curated D3 examples beyond basics; shows custom interactions. ([Observable](#))
- **Apache ECharts Examples** – rich, interactive charts (maps, timelines, large datasets). ([ECharts](#))
- **USGS Latest Earthquakes** – live, interactive map + filters; perfect data-map pattern. ([USGS Earthquake Hazards](#))
- **Leaflet Examples/Quick Start** – interactive map basics and patterns to copy. ([Leaflet](#))
- **MapLibre GL JS Examples** – WebGL vector-tile maps with hover, clustering, sliders, etc. ([MapLibre](#))

Games / Entertainment (Canvas/WebGL engines)

- **Phaser 3 Examples** – hundreds of runnable game snippets (input, physics, shaders, scenes). ([Phaser](#))
- **Phaser Labs** – live playground for the newest Phaser builds. ([Phaser Labs](#))
- **PixiJS Examples** – WebGL renderer demos (sprites, filters, text, tiling, etc.). ([PixiJS](#))